

LAB REPORT: Pointers in C++

Title: Pointer Declaration, Initialization and Pointer Arithmetic in C++

1. Pointer Declaration and Initialization

Objective

To declare an integer variable and a pointer, initialize the pointer with the address of the variable, and display values using the dereference operator.

Theory

A **pointer** is a variable that stores the memory address of another variable. The * (dereference operator) is used to access the value stored at that address.

- & → address of operator
 - * → value at address (dereference)
-

Program Code

```
#include <iostream>
using namespace std;

int main() {

    // Step 1: Declare and initialize integer variable
    int num = 50;

    // Step 2: Declare pointer variable
    int *ptr;

    // Step 3: Store address of num in pointer
    ptr = &num;

    // Step 4: Display values
    cout << "Value of num: " << num << endl;
    cout << "Address of num: " << &num << endl;
    cout << "Value using pointer (*ptr): " << *ptr << endl;

    return 0;
}
```

Explanation (Step by Step)

- `int num = 50;` → normal variable
- `int *ptr;` → pointer declaration
- `ptr = #` → pointer stores address of num
- `*ptr` → gives actual value stored at that address

Expected Output

Value of num: 50
Address of num: 0x61ff08 (example)
Value using pointer (*ptr): 50

Conclusion

This program demonstrates how pointers store memory addresses and access values using the dereference operator.

2. Pointer Arithmetic

Objective

To use pointers to access array elements using pointer arithmetic.

Theory

Pointer arithmetic allows us to move between array elements using a pointer.

- `ptr + 1` → next element
- `*(ptr + i)` → i-th element value

Arrays and pointers are closely related in C++.

Program Code

```
#include <iostream>
using namespace std;

int main() {

    // Step 1: Declare and initialize array
    int numbers[] = {10, 20, 30, 40, 50};

    // Step 2: Pointer to first element of array
    int *ptr = numbers;

    // Step 3: Access elements using pointer arithmetic
    cout << "Array elements using pointer arithmetic:" << endl;

    for(int i = 0; i < 5; i++) {
        cout << "Element " << i << " = " << *(ptr + i) << endl;
    }

    return 0;
}
```

Explanation (Step by Step)

- `int numbers[] = {...};` → array created
- `int *ptr = numbers;` → pointer points to first element
- `ptr + i` → moves to next memory location
- `*(ptr + i)` → prints value at that position

Expected Output

Array elements using pointer arithmetic:

Element 0 = 10

Element 1 = 20

Element 2 = 30

Element 3 = 40

Element 4 = 50

Conclusion

This program shows how pointers can be used to access array elements efficiently using pointer arithmetic.

Final Summary

Task	Concept Used	Key Idea
Pointer Basics	Address + Dereference	* and &
Pointer Arithmetic	Array Traversal	<code>ptr + i</code>